

AJX 动效的前世今生

动效是什么？它重要吗？

其实动效你每天都在接触，比如你在购物 APP 上买东西，物品收进购物车时的收起动画。再比如你当前所看到这篇教程所在的网页，当你点击最小化时，它的收起动画。

程序中的动效随处可见，而且它必不可少，它是让用户体验和感受升级的有效手段。也是一个优秀网页、APP、游戏的标配。

动效等于动画吗？

动效不完全等于动画，实际上动效可以通过动画、视频、gif 图等方式实现。笔者认为准确来说动效应该包含两种：

1. 播放类动效
2. 交互类动效

通常来说，播放类动效都是较为简单明确的，例如下面这个动画：



normal_animate

它是通过改变图形的宽度和高度来实现的，如果再加上移动还会更好玩：



normal_animate2

那么交互类动画是什么呢？让我们直接看一个例子：



drag_rect

区分播放类还是交互类动效的关键在于动效的执行过程中「是否存在用户的手势操作」。实际上上图中的交互类动效中还包含了2个播放类动效，你发现了吗？

AJX 的动效发展历史

历史上，AJX 经历了 [三个时期](#) 的迭代。

🔥 第一个时期

在第一个 AJX 时期，只有 XML 模版来映射 UI，那时候即使你想要最简单的动效都是无法实现的。那一时期的重点在于用 XML 来描述 UI 布局。如果涉及到动效的需求是需要绕开或者由原生实现的。

💡 第二个时期

在第二个 AJAX 时期，AJAX 在 XML 模版 UI 的基础上拥有了 Javascript 运行环境，可以开发相对复杂的业务逻辑。这一时期，早期的属性动画 API 产生了：

▼

复制代码

```
//创建动画对象
let animation = new AJAXAnimation();

//设置执行动画目标view
animation.setTarget(ui.find("animTarget"));

//设置执行动画的时间
animation.setDuration(500);

//设置执行动画的速率（0：线性匀速，1：减速，2：加速）
animation.setInterpolator(2);

//设置开始时属性
animation.setStartParams({'translatey': 100});

//设置结束时属性
animation.setEndParams({'translatey': -100});

//执行动画
animation.start();

//监听动画回调
animation.onStateChange( function(state){
    if(state == 0){
        print("animation Start");
    }
    else{
        print("animation end");
    }
});
```

之前做过一次动画的分享，感兴趣的同学可以下载看看，[动态 UI 动画与列表滚动优化分享\(2016.7.4\).key](#)。

在这一时期，能够完成播放类动效和弱交互类动效。由于这一时期 AJAX 只有主线程运行环境，并且在第二代 AJAX 中布局是采用与安卓相对布局相同的布局思想实现的。属性动画的变化会影响布局，并且由于在主线程中不可做大量动画逻辑操作以免耗时任务阻塞 UI 线程，所以第二代属性动画的 API 比较单一。

第三个时期

第三个 AJAX 时期，动效能力百花齐放，这些能力有：

- 视频
- gif 图
- canvas 绘制
- Hilo 游戏引擎
- 属性动画
- 队列动画
- 关联动画
- lottie 动画等
- f2 数据可视化

目前我们处于第三代 AJAX 时期，实际上从第三代开始，大量的成熟技术应用在 AJAX 中，对动效产生最大影响的有以下两点：

- 布局改变，切换到更贴近前端体验的 Flex 布局，布局逻辑下沉到 C++ 层，保证双端布局效果一致
- 线程模型改变，由单一的主线程运行环境改变成主线程 + JS 线程

磨刀不误砍柴功

AJAX 动效历史你清楚了，但是我们先不要急于学习 API 的使用，如果你准备深入理解并灵活运用 AJAX 中的动效技术，先搞懂最关键的布局系统和线程模型。

布局系统

在动态 UI 中，界面的元素节点是通过 XML 语言来描述的，最终组织成节点的树形结构，即 [DOM](#)。可以通过 inflate 接口直接构建一个页面结构。

`ajax.ui.inflate(xml)` 将字符串转换为 Node 节点

[CSS](#) 是 AJAX 目前使用的样式表语言，布局采用流行的 [盒模型](#) + [Flex](#) 布局。

注意了，下面我将说明在 AJAX 中进行动画或者动效开发的关键问题。

如何理解 DOM？要从 AJAX 的绘制原理讲起，AJAX 相对于其他跨平台方案高性能的原因是由于它享受的是原生组件的能力。

而原生能力是由谁提供的呢？要运行在 Android 和 iOS 势必要提供这两端的原生控件和逻辑的实现，通常将其称为「AJAX 适配层」。

你可以理解为有一套大的通用协议接口，由多端的适配层来分别实现协议的接口。而这套接口的定义以及通用逻辑是由一个 C++ 引擎来实现的达到多端复用和高度统一的，通常将其称为「AJAX 引擎层」。

而在「AJAX 适配层」和「AJAX 引擎层」之上的才是离我们开发者最近的「AJAX 业务层」。

所以我们的绘制流程是：

业务层描述 XML 和 CSS 样式表 -> 引擎层翻译成盒模型和Flex布局并建立 DOM 树 -> 适配层接收引擎接口 DOM 信息后创建并更新 UI 视图。

[框架原理](#)大致如下图：



这三层关系是你在 AJAX 开发时经常会听到的，建议你一定要烂熟于胸。因为这一般是 AJAX

支撑同学的口头用语，知道了这个分层关系的你，锅才能顺利落地  beiguo。

例如避免如下尴尬：

OO 同学：“XX 同学，我的 Flex 布局按照文档写的，你知道为啥我得到的结果不对吗？”

XX 同学：“OO 同学，你说这个问题是由AJX C++引擎实现的，建议你找下他们。”

OO 同学：“WW 同学，XX 同学说我这个 Flex 布局的问题应该找你。”

WW 同学：“OO 同学，你有 Demo 吗？”

OO 同学：“我刚写了个 Demo 没问题，哦，我好像少写了一个字母 ‘t’。”

... ..

有点扯远了，现在我们应该对这三层关系理解又进了一步。那么聪明的你有没有发现什么问题呢？

如果我们通过动画只改变了适配层的 View ，引擎层的 DOM 结构会更新吗？或者反过来呢？

敲黑板了：引擎层的 DOM 和适配层的 View 在有动画存在时，会产生不同步。如果在动画过程中实时同步，会带来较大的双向通信负担。所以需要我们在开发时保证动画在结束状态或合理的时机同步 DOM 和 View 的属性。

线程模型

AJX 的线程一共有2条：

- UI线程 - 适配层运行环境
- JS线程 - 引擎层与业务层运行环境

前端业务层和适配层之间不能直接交互，都需要通过引擎层做转接，前端不感知UI线程，适配层不感知JS线程。

再敲下黑板：由于双线程存在，切换线程不可避免，但我们应该尽量避免线程交互，避免线程交互的方法在后续文章中会逐一讲解。

动效的术与道

从动效的理解和重要性，到 AJX 动效的发展历史，再到 AJX 的框架结构和动效的大前提。是从道到术的应用。

何为术？

这篇教程80%以上的部分是讲解的 AJX 动效开发的术。它教你如何使用 AJX 中的 API 和帮助你理解其作用。就像一篇武功秘籍的外功，它招式繁多，个个有用。

何为道？

动效开发，不仅仅存在于 AJX 中，更存在于更多的场景和平台。动效的核心在于道，道是什么？它是你的经验，或是你的思维模式。设计一个动效，需要你更多理解物理世界和人的感官世界。实现一个动效，需要你对它进行分解、排序、整合。道的修炼就像这篇武功秘籍的内功，在你的学习和应用过程中逐渐加深、升华。道没有捷径，如果有，那就是让你的大脑思考更多。

👉，现在你将正式开始学习 AJX 动效的一招一式，[一元复始，属性动画再现江湖](#)。